## Contents

**COMPETITIVE PROGRAMMING MASTERY ROADMAP1Zero to Hero: Dominate Codeforces, AtCoder & Beyond11. THE COMPETITIVE PROGRAMMING LANDSCAPE1*What is Competitive Programming?1Why Competitive Programming?1CP Rating Systems1The CP Mindset22. PHASE 0: GETTING STARTED2Duration: 1 week2Setup & Tooling2Platform Registration3Understanding Time Limits & Input Sizes33. PHASE 1: FOUNDATIONS & STL MASTERY3Duration: 2-3 weeks33.1 C++ STL Containers43.2 STL Algorithms53.3 Big-O Analysis & Complexity64. PHASE 2: CORE DSA FOR CP6Duration: 6-8 weeks64.1 Arrays & Strings64.2 Recursion & Backtracking74.3 Binary Search74.4 Linked Lists84.5 Stacks & Queues84.6 Trees & Graphs84.7 Dynamic Programming94.8 Bit Manipulation105. PHASE 3: INTERMEDIATE TOPICS10Duration: 4-6 weeks105.1 Number Theory105.2 Combinatorics115.3 Range Query Data Structures125.4 Graph Algorithms Advanced135.5 String Algorithms146. PHASE 4: ADVANCED TOPICS15Duration: 6-8 weeks156.1 Heavy-Light Decomposition156.2 Centroid Decomposition166.3 Link-Cut Tree166.4 Flow Algorithms Advanced166.5 Convex Hull Trick & Li Chao Tree186.6 FFT & NTT196.7 Game Theory196.8 Polynomial & String Hashing207. PHASE 5: EXPERT & MASTER LEVEL20Duration: 8-12+ weeks207.1 Advanced Data Structures207.2 Geometry & Computational Geometry227.3 Complexity Theory238. PLATFORM STRATEGIES238.1 Codeforces Strategy238.2 AtCoder Strategy248.3 CodeChef Strategy248.4 LeetCode Strategy259. CONTEST STRATEGY & TEMPLATES259.1 Pre-Contest Preparation259.2 During Contest269.3 Post-Contest269.4 Template Code269.5 Stress Testing2810. 100+ MUST-SOLVE PROBLEMS2910.1 Arrays & Prefix Sums (10 problems)2910.2 Binary Search (10 problems)2910.3 Stacks & Queues (10 problems)2910.4 Linked Lists (8 problems)3010.5 Trees & BST (12 problems)3010.6 Graphs (15 problems)3010.7 Dynamic Programming (20 problems)3110.8 Bit Manipulation (8 problems)3110.9 Number Theory & Combinatorics (10 problems)3210.10 Advanced (15+ problems)3211. RATING TIERS & SKILLS TABLE33Equivalent Ratings Across Platforms3312. RESOURCES & TOOLS33Must-Visit Websites33Must-Follow People34Essential Books34Tools & Extensions35ICPC Preparation35FINAL WORDS36***

# COMPETITIVE PROGRAMMING MASTERY ROADMAP

### Zero to Hero: Dominate Codeforces, AtCoder & Beyond

**Current Status (June 2026):** - Codeforces: Unrated | 0 problems - AtCoder: Unrated | 0 problems - CodeChef: Unrated | 0 problems - LeetCode: 53 solved | Rank ~2.5M - Language: C++

---

## 1. THE COMPETITIVE PROGRAMMING LANDSCAPE

### What is Competitive Programming?

Competitive programming is a mind sport where participants solve well-defined algorithmic problems under time constraints. It tests problem-solving ability, algorithmic knowledge, and coding speed.

### Why Competitive Programming?

```
- Top tech companies (Google, Meta, Microsoft) value CP achievements
- ICPC World Finalists get direct interview calls
- Develops speed and accuracy in problem-solving
- Builds strong algorithmic intuition
- Community: global network of talented programmers
- Opens doors to research and high-frequency trading roles
```

### CP Rating Systems

| Platform | Rating Range | Lowest | Highest |
|---|---|---|---|
| Codeforces | 0-4000+ | Newbie (0-1199) | Legendary Grandmaster (3000+) |
| AtCoder | 0-4000+ | Gray (0-399) | Red (2800+) |
| CodeChef | 0-4000+ | 1 Star (0-1399) | 7 Star (2500+) |
| LeetCode | 0-3500+ | 1400-1599 | Guardian (2200+) |

### The CP Mindset

| Casual Programmer | Competitive Programmer |
|---|---|
| Solves when free | Practices daily, no excuses |
| Avoids hard problems | Hunts down hard problems |
| Skips editorials | Devours editorials |
| One language okay | Masters C++ STL |
| No contest participation | Lives for contests |
| No timer | Tracks every minute |

## 2. PHASE 0: GETTING STARTED

### Duration: 1 week

### Setup & Tooling

```
Compiler & IDE Setup:
  - GCC/G++ with C++17/20 support
  - VS Code with CP extensions:
    - Competitive Programming Helper (cph)
    - C++ TestMate
    - Code Runner
    - Better Comments
  - CP Editor (standalone, lightweight)
  - Terminal aliases for quick compilation

Compilation Flags:
  g++ -std=c++17 -O2 -Wall -Wextra -Wshadow -DONLINE_JUDGE
       -Wno-unused-result -Wno-sign-compare
  -O2: optimizations for speed
  -Wall -Wextra: enable warnings
  -Wshadow: catch variable shadowing bugs
  -DONLINE_JUDGE: define macro (optional debug prints)

Fast I/O (required for CP):
  #include <bits/stdc++.h>
  using namespace std;

  int main() {
      ios_base::sync_with_stdio(false);
      cin.tie(NULL);
      cout.tie(NULL);

      int t; cin >> t;
      while (t--) {
          // solve
      }
```

```
      return 0;
  }

Debugging Setup:
  #define dbg(x) cerr << #x << " = " << x << '\n'
  #define dbga(arr, n) for(int i=0;i<n;i++) cerr<<arr[i]<<" \n"[i==n-1]

  // Local debug (not printed on judge)
  #ifndef ONLINE_JUDGE
      freopen("input.txt", "r", stdin);
      freopen("output.txt", "w", stdout);
      freopen("error.txt", "w", stderr);
  #endif

Template File (template.cpp):
  - Hold this in a file you copy for every contest
  - Includes: all STL headers, fast I/O, debug macros
  - Type aliases: ll (long long), vll, vi, vvi, pii
  - Modulo constants: MOD = 1e9+7
  - Common functions: binpow, gcd, lcm, isPrime
```

### Platform Registration

```
Create accounts on:
  1. Codeforces: cf.codeforces.com ★ Primary
  2. AtCoder: atcoder.jp ★ Secondary
  3. CodeChef: codechef.com ★ Practice
  4. LeetCode: leetcode.com ★ Interview-focused
  5. SPOJ: spoj.com ★ Classic problems
  6. CSES: cses.fi ★ Excellent problem set

Set up profiles:
  - Profile picture (professional or clean)
  - Country, institution
  - Consistent handle across platforms
  - Link to GitHub
```

### Understanding Time Limits & Input Sizes

```
Input Size → Required Complexity:
  n ≤ 10: O(n!) factorial, brute force
  n ≤ 20: O(2^n) subset enumeration
  n ≤ 100: O(n³) Floyd-Warshall, DP
  n ≤ 1000: O(n²) bubble sort, 2D DP
  n ≤ 10⁵: O(n log n) sort, segment tree, Dijkstra
  n ≤ 10⁶: O(n) linear scan, DP
  n ≤ 10⁷: O(n log n) with optimization
  n ≤ 10⁹: O(log n) binary search, fast exponentiation

Time Limit Equivalence (1 second):
  10⁸ operations: approximate limit in C++
  With O2 optimization: ~2-5 × 10⁸ simple operations
  Division, modulo: slower than addition/multiplication
  STL overhead: vectors are fast, maps slower, unordered_maps may collide
```

---

## 3. PHASE 1: FOUNDATIONS & STL MASTERY

### Duration: 2-3 weeks

### 3.1 C++ STL Containers

```
Sequence Containers:

  vector<T>:
    - Dynamic array, O(1) amortized push_back, O(n) insert/erase
    - reserve(n): pre-allocate memory
    - shrink_to_fit(): release excess memory
    - emplace_back(args...): construct in-place (faster than push_back)
    - Methods: size, empty, clear, front, back, data
    - Iterators: begin, end, rbegin, rend

  deque<T>:
    - Double-ended queue, O(1) push/pop both ends
    - Random access O(1), but slightly slower than vector
    - Use: sliding window maximum, BFS

  list<T> (doubly linked list):
    - O(1) insert/erase anywhere (if you have iterator)
    - No random access, high memory overhead
    - splice: O(1) transfer elements between lists
    - Use: when frequent middle insertions needed

  array<T, N> (C++11):
    - Fixed-size array, stack-allocated
    - No dynamic allocation overhead
    - Use: when size known at compile time

Container Adaptors:
  stack<T>: LIFO, push, pop, top
  queue<T>: FIFO, push, pop, front, back
  priority_queue<T>: max-heap by default, push, pop, top

  priority_queue customization:
    priority_queue<int, vector<int>, greater<int>> pq; // min-heap
    priority_queue<pair<int,int>, vector<pair<int,int>>, greater<>> pq;

Associative Containers (ordered):
  set<T>: sorted, unique elements, O(log n) operations
  multiset<T>: sorted, allows duplicates
  map<K, V>: sorted key-value pairs, O(log n)
  multimap<K, V>: sorted, allows duplicate keys

  IMPORTANT: set and map iterators invalidate differently from vector
  - set::insert doesn't invalidate iterators
  - map::erase invalidates only the erased element

Unordered Containers:
  unordered_set<T>: hash table, O(1) average, O(n) worst
  unordered_map<K, V>: hash table, O(1) average
  unordered_multiset, unordered_multimap

  Custom hash for pairs/tuples:
    struct pair_hash {
        size_t operator()(const pair<int,int>& p) const {
            return hash<int>()(p.first) ^ (hash<int>()(p.second) << 1);
        }
    };
    unordered_set<pair<int,int>, pair_hash> s;

Other Containers:
```

```
  bitset<N>: fixed-size bit array, operations in O(N/word)
    - count, any, all, none, set, reset, flip
    - Bitwise: &, |, ^, ~, <<, >>
    - Use: subset DP, bit operations

  string: specialized container for characters
    - find, rfind, substr, compare, +=, + operator
    - c_str() for C-style string
    - stoi, stoll, to_string conversions
    - stringstream for parsing
```

### 3.2 STL Algorithms

```
Non-modifying:
  all_of, any_of, none_of: check conditions
  find, find_if, find_if_not: linear search
  count, count_if: count occurrences
  equal: compare two ranges
  search: find subsequence
  min_element, max_element, minmax_element: extremum
  adjacent_find: find adjacent equal elements

Modifying:
  copy, copy_if, copy_n: copy elements
  fill, fill_n: assign value
  transform: apply function to elements
  replace, replace_if: replace elements
  remove, remove_if: remove by condition (shifts, doesn't erase)
  unique: remove consecutive duplicates
  reverse: reverse order
  rotate: rotate around middle element
  shuffle: random shuffle (C++11, with random engine)

Sorting & Partitioning:
  sort: O(n log n) introsort (quicksort + heapsort)
  stable_sort: O(n log n) mergesort, stable
  partial_sort: only sort first k elements O(n log k)
  nth_element: find nth position element O(n) average
  partition, stable_partition: partition by predicate
  is_sorted, is_sorted_until: check sorting

Binary Search (on sorted ranges):
  lower_bound: first element ≥ value (O(log n))
  upper_bound: first element > value (O(log n))
  equal_range: pair of lower_bound and upper_bound
  binary_search: returns bool

  // Usage on vector
  auto it = lower_bound(v.begin(), v.end(), x);
  int idx = it - v.begin();

  // On sets/maps (use member functions for O(log n))
  auto it = s.lower_bound(x);  // O(log n)

Heap Operations:
  make_heap: build heap from range O(n)
  push_heap: add element to heap O(log n)
  pop_heap: remove top element O(log n)
  sort_heap: convert heap to sorted array O(n log n)

Set Operations (on sorted ranges):
```

```
  merge: merge two sorted ranges
  includes: check if one range is subset of another
  set_union, set_intersection, set_difference, set_symmetric_difference

Min/Max:
  min, max, minmax: for two values or initializer list
  clamp (C++17): clamp(x, lo, hi)

Numeric:
  accumulate: sum of elements
  inner_product: dot product of two sequences
  adjacent_difference: differences between adjacent elements
  partial_sum: cumulative sums (prefix sums)
  iota: fill with increasing values

Permutations:
  next_permutation, prev_permutation: generate all permutations
  is_permutation: check if range is permutation of another
```

### 3.3 Big-O Analysis & Complexity

```
Formal Definitions:
  O(g(n)) = { f(n): ∃ c > 0, n₀ > 0 such that 0 ≤ f(n) ≤ c·g(n) for all n ≥ n₀ }
  Ω(g(n)) = { f(n): ∃ c > 0, n₀ > 0 such that 0 ≤ c·g(n) ≤ f(n) for all n ≥ n₀ }
  Θ(g(n)) = O(g) ∩ Ω(g)

Common Complexity Classes:
  O(1): constant - array access, hash table lookup (average)
  O(log n): logarithmic - binary search, balanced BST
  O(√n): square root - primality test, divisor enumeration
  O(n): linear - linear search, prefix sum
  O(n log n): linearithmic - comparison sort, divide & conquer
  O(n²): quadratic - simple sort, nested loops on entire input
  O(n³): cubic - Floyd-Warshall, matrix multiplication (naive)
  O(2ⁿ): exponential - subset enumeration
  O(n!): factorial - permutation enumeration

Amortized Analysis:
  - vector push_back: O(1) amortized (O(n) worst-case single operation)
  - deque push_front: O(1) amortized
  - splay tree operations: O(log n) amortized

Space Complexity:
  - Always mention space alongside time in solutions
  - In-place: O(1) extra space
  - Recursion: O(depth) stack space
  - Common tradeoffs: time-space tradeoff (e.g., memoization)
```

---

## 4. PHASE 2: CORE DSA FOR CP

### Duration: 6-8 weeks

### 4.1 Arrays & Strings

```
Key Techniques:
  - Two pointers (sliding window)
  - Prefix sums (1D, 2D)
  - Difference arrays (range updates)
  - Kadane's algorithm (max subarray)
  - Dutch National Flag (3-way partition)
```

```
  - Manacher's algorithm (palindromes)
  - Z-algorithm & KMP (pattern matching)
  - Rabin-Karp (rolling hash)

Common Problems:
  - Subarray sum equals K
  - Longest substring without repeating characters
  - Container with most water
  - Trapping rain water
  - Next permutation
  - Rotate array/image
```

### 4.2 Recursion & Backtracking

```
Pattern:
  void solve(state, parameters) {
      if (base_case) {
          process_answer();
          return;
      }
      for (each_choice) {
          if (is_valid(choice)) {
              make_move(choice);
              solve(new_state, updated_params);
              undo_move(choice);
          }
      }
  }

Key Problems:
  - Generate all subsets / permutations / combinations
  - N-Queens, Sudoku Solver
  - Rat in a Maze
  - M-Coloring Problem
  - Knight's Tour
  - Palindrome Partitioning
  - Subset Sum / Combination Sum
  - Word Search / Boggle

Optimization Tips:
  - Pruning: early exit for invalid branches
  - Ordering: try promising branches first
  - Bitmask state representation
  - Memoization (recursion + caching = DP)
```

### 4.3 Binary Search

```
Beyond Simple Search:
  - Binary search on answer (monotonic predicate)
  - Find peak in array (mountain array)
  - Search in rotated sorted array
  - Find minimum in rotated sorted array
  - Aggressive cows / Book allocation (minimize max)
  - Koko eating bananas / Capacity to ship packages
  - Nth root of a number
  - Median of two sorted arrays
  - Kth element in sorted matrix

  Pattern:
    while (lo <= hi) {
        int mid = lo + (hi - lo) / 2;
```

```
        if (possible(mid)) hi = mid - 1; // or lo = mid + 1
        else lo = mid + 1;               // direction depends on problem
    }
```

### 4.4 Linked Lists

```
Operations:
  - Reverse (iterative, recursive, in K groups)
  - Detect cycle (Floyd's cycle detection)
  - Find intersection point
  - Remove Nth from end
  - Merge sorted lists
  - LRU Cache (doubly linked list + hashmap)
  - Clone with random pointers
  - Palindrome check
  - Flatten a multilevel linked list

Two Pointer Patterns:
  - Slow and fast pointers (cycle detection, middle element)
  - List1 and List2 pointers (intersection, merging)
  - Current and Prev pointers (reversal)
```

### 4.5 Stacks & Queues

```
Monotonic Stack Pattern (critical for CP):
  - Next Greater Element (right, left)
  - Previous Greater/Smaller Element
  - Largest Rectangle in Histogram
  - Maximal Rectangle (2D extension)
  - Trapping Rain Water
  - Sum of Subarray Minimums/Maximums
  - Asteroid Collision
  - Remove K Digits (smallest possible)
  - Stock Span Problem

  For "Next Greater Element":
    stack<int> st;
    vector<int> nge(n, -1);
    for (int i = n-1; i >= 0; i--) {
        while (!st.empty() && arr[st.top()] <= arr[i])
            st.pop();
        if (!st.empty())
            nge[i] = arr[st.top()];
        st.push(i);
    }

Deque Applications:
  - Sliding Window Maximum/Minimum
  - Find maximum in all subarrays of size K
  - Sliding window with conditions
```

### 4.6 Trees & Graphs

```
Tree Traversals:
  - Inorder, Preorder, Postorder (recursive + iterative)
  - Level order (BFS)
  - Morris traversal (O(1) space)
  - Vertical order, Top view, Bottom view

Tree Properties:
  - Height, diameter, max path sum
```

```
  - Balanced tree check
  - LCA (binary lifting, Euler tour + RMQ)
  - Serialize / Deserialize
  - Segment Tree, Fenwick Tree (for range queries)

Graph Representations:
  - Adjacency matrix: O(V²) memory, O(1) edge check
  - Adjacency list: O(V+E) memory, O(deg) neighbors
  - Edge list: simple, used in Kruskal's

Graph Algorithms:
  BFS: shortest path (unweighted), topological sort (Kahn's)
  DFS: connectivity, cycle detection, bipartite check
  Dijkstra: shortest path (weighted, non-negative)
  Bellman-Ford: shortest path (negative weights allowed)
  Floyd-Warshall: all-pairs shortest (O(V³))
  Prim's / Kruskal's: Minimum Spanning Tree
  DSU (Disjoint Set Union): connectivity, Kruskal's

  DSU Implementation:
    struct DSU {
        vector<int> parent, size;
        DSU(int n) {
            parent.resize(n);
            size.assign(n, 1);
            iota(parent.begin(), parent.end(), 0);
        }
        int find(int x) {
            return parent[x] == x ? x : parent[x] = find(parent[x]);
        }
        void unite(int a, int b) {
            a = find(a); b = find(b);
            if (a == b) return;
            if (size[a] < size[b]) swap(a, b);
            parent[b] = a;
            size[a] += size[b];
        }
    };
```

### 4.7 Dynamic Programming

```
DP Approaches:
  1. Memoization (top-down): recursive + cache
  2. Tabulation (bottom-up): iterative, fill table
  3. Space optimization: reduce dimension (when possible)

Classic DP Patterns:
  Fibonacci-style: 1D DP, state = position
  Grid DP: unique paths, min path sum
  Subset Sum / KnapSack: 0/1 and unbounded
  LIS (Longest Increasing Subsequence): O(n log n) with patience sorting
  LCS (Longest Common Subsequence): 2D DP on strings
  Edit Distance: string transformation
  Matrix Chain Multiplication: MCM pattern (interval DP)
  DP on Trees: subtree computations
  DP on Bitmask: traveling salesman, subset cover
  DP with Digit: count numbers with given property

  DP Optimization Techniques:
    - Prefix sums (for transition optimization)
    - Convex Hull Trick (for linear DP)
```

```
    - Divide & Conquer DP (for monotonic decision points)
    - Knuth optimization (quadrangle inequality)
    - Matrix exponentiation (for linear recurrences)
```

### 4.8 Bit Manipulation

```
Essential Bit Tricks:
  x & (x-1): clear lowest set bit
  x & -x: extract lowest set bit
  x | (x-1): set all bits after lowest zero
  ~x: complement
  x ^ x: 0 (a ^ a = 0)
  x ^ 0: x

Common Patterns:
  - Check if power of two: x > 0 && (x & (x-1)) == 0
  - Count set bits: __builtin_popcount(x)
  - Count leading zeros: __builtin_clz(x)
  - Count trailing zeros: __builtin_ctz(x)
  - Enumerate subsets: for(int sub = mask; sub; sub = (sub-1) & mask)
  - Isolate LSB: x & -x
  - Check if kth bit set: (x >> k) & 1
  - Set kth bit: x |= (1 << k)
  - Unset kth bit: x &= ~(1 << k)
  - Toggle kth bit: x ^= (1 << k)
```

---

## 5. PHASE 3: INTERMEDIATE TOPICS

### Duration: 4-6 weeks

### 5.1 Number Theory

```
Primality:
  - Sieve of Eratosthenes: O(n log log n)
  - Linear sieve (SPF): smallest prime factor, O(n)
  - Segmented sieve: for large ranges (L, R)
  - Miller-Rabin: probabilistic O(k log n)
  - Pollard Rho: factorization O(n^(1/4))

  Linear sieve:
    vector<int> primes, spf(N+1);
    for (int i = 2; i <= N; i++) {
        if (spf[i] == 0) {
            spf[i] = i;
            primes.push_back(i);
        }
        for (int p : primes) {
            if (p > spf[i] || i * p > N) break;
            spf[i * p] = p;
        }
    }

GCD & Friends:
  - Euclid's algorithm: gcd(a,b) = gcd(b, a%b)
  - Extended Euclid: ax + by = gcd(a,b)
  - LCM: a / gcd(a,b) * b

Modular Arithmetic:
  - Modular addition: (a + b) % MOD
  - Modular multiplication: (a * b) % MOD
```

```
  - Modular exponentiation: binpow(a, b, MOD)
  - Modular inverse: a^(-1) mod M (Fermat: a^(M-2) if M prime)
  - Precompute factorials + inverses for nCr

  binpow:
    ll binpow(ll a, ll b, ll mod) {
        ll res = 1;
        while (b > 0) {
            if (b & 1) res = (res * a) % mod;
            a = (a * a) % mod;
            b >>= 1;
        }
        return res;
    }

  nCr with precomputation:
    vector<ll> fact(N+1), invfact(N+1);
    fact[0] = 1;
    for (int i = 1; i <= N; i++) fact[i] = fact[i-1] * i % MOD;
    invfact[N] = binpow(fact[N], MOD-2, MOD);
    for (int i = N-1; i >= 0; i--) invfact[i] = invfact[i+1] * (i+1) % MOD;

    ll nCr(int n, int r) {
        if (r < 0 || r > n) return 0;
        return fact[n] * invfact[r] % MOD * invfact[n-r] % MOD;
    }

Chinese Remainder Theorem:
  - Solve x ≡ a₁ (mod m₁), x ≡ a₂ (mod m₂), ...
  - Requires coprime moduli (generalized with Garner's algorithm)

Totient & Möbius:
  - φ(n): count of integers ≤ n coprime to n
  - μ(n): Möbius function (0 if square factor, (-1)^k otherwise)
  - Linear sieve for both
```

### 5.2 Combinatorics

```
Counting Fundamentals:
  - P(n,k) = n! / (n-k)! (permutations)
  - C(n,k) = n! / (k! (n-k)!) (combinations)
  - Stars and Bars: distributing identical items
  - Inclusion-Exclusion Principle
  - Derangements: !n = (n-1)(!(n-1) + !(n-2))
  - Catalan numbers: C_n = (1/(n+1)) * C(2n, n)

Generating Functions:
  - Ordinary generating functions: Σ a_n x^n
  - Exponential generating functions: Σ a_n x^n / n!
  - Product of generating functions: convolution of sequences

Recurrence Solving:
  - Master theorem for divide & conquer recurrences
  - Characteristic equation for linear recurrences
  - Matrix exponentiation for fast nth term

  Matrix exponentiation:
    using Matrix = vector<vector<ll>>;
    Matrix mat_mul(Matrix a, Matrix b, ll mod) {
        int n = a.size(), m = b[0].size(), p = a[0].size();
        Matrix c(n, vector<ll>(m));
```

```
        for (int i = 0; i < n; i++)
            for (int k = 0; k < p; k++)
                if (a[i][k])
                    for (int j = 0; j < m; j++)
                        c[i][j] = (c[i][j] + a[i][k] * b[k][j]) % mod;
        return c;
    }

    Matrix mat_pow(Matrix a, ll b, ll mod) {
        int n = a.size();
        Matrix res(n, vector<ll>(n));
        for (int i = 0; i < n; i++) res[i][i] = 1;
        while (b) {
            if (b & 1) res = mat_mul(res, a, mod);
            a = mat_mul(a, a, mod);
            b >>= 1;
        }
        return res;
    }
```

### 5.3 Range Query Data Structures

```
Fenwick Tree (Binary Indexed Tree):
  - Point update, prefix sum query: O(log n)
  - Range update + point query: use difference array
  - Range update + range query: two BITs
  - Cannot handle min/max (use segment tree)

  Implementation:
    struct BIT {
        vector<ll> bit;
        int n;
        BIT(int n) : n(n), bit(n+1) {}
        void add(int idx, ll delta) {
            for (; idx <= n; idx += idx & -idx)
                bit[idx] += delta;
        }
        ll sum(int idx) {
            ll res = 0;
            for (; idx > 0; idx -= idx & -idx)
                res += bit[idx];
            return res;
        }
        ll range(int l, int r) { return sum(r) - sum(l-1); }
    };

Segment Tree:
  - Build: O(n), Query: O(log n), Update: O(log n)
  - Supports: sum, min, max, gcd, and (custom combine)
  - Range update: lazy propagation

  Implementation (iterative):
    struct SegTree {
        vector<ll> tree;
        int n;
        SegTree(vector<ll>& arr) {
            n = arr.size();
            tree.resize(2*n);
            for (int i = 0; i < n; i++) tree[n+i] = arr[i];
            for (int i = n-1; i > 0; i--) tree[i] = tree[2*i] + tree[2*i+1];
        }
```

```
        void update(int idx, ll val) {
            idx += n;
            tree[idx] = val;
            for (idx /= 2; idx >= 1; idx /= 2)
                tree[idx] = tree[2*idx] + tree[2*idx+1];
        }
        ll query(int l, int r) {
            l += n; r += n;
            ll res = 0;
            while (l <= r) {
                if (l & 1) res += tree[l++];
                if (!(r & 1)) res += tree[r--];
                l /= 2; r /= 2;
            }
            return res;
        }
    };

Sparse Table:
  - Precompute: O(n log n), Query: O(1)
  - Supports idempotent operations (min, max, gcd)
  - No updates allowed

  vector<vector<ll>> st(N, vector<ll>(K+1));
  for (int i = 0; i < n; i++) st[i][0] = arr[i];
  for (int j = 1; j <= K; j++)
      for (int i = 0; i + (1<<j) <= n; i++)
          st[i][j] = min(st[i][j-1], st[i + (1<<(j-1))][j-1]);

  int k = __lg(r-l+1);
  ll ans = min(st[l][k], st[r-(1<<k)+1][k]);

Mo's Algorithm:
  - Offline query processing: O((N+Q)√N)
  - Sort queries by (block, r) → add/remove pointers
  - Good for: range frequency, distinct count, inversions
```

### 5.4 Graph Algorithms Advanced

```
Strongly Connected Components:
  - Kosaraju's: 2 DFS passes (forward + reverse graph)
  - Tarjan's: single DFS using lowlink values

  int timer = 0;
  vector<int> tin(N, -1), low(N), comp(N, -1);
  stack<int> st;
  vector<bool> instack(N);

  void dfs(int v) {
      tin[v] = low[v] = timer++;
      st.push(v); instack[v] = true;
      for (int u : graph[v]) {
          if (tin[u] == -1) { dfs(u); low[v] = min(low[v], low[u]); }
          else if (instack[u]) low[v] = min(low[v], tin[u]);
      }
      if (low[v] == tin[v]) {
          while (true) {
              int u = st.top(); st.pop(); instack[u] = false;
              comp[u] = v;
              if (u == v) break;
          }
```

```
      }
  }

Bridges & Articulation Points:
  - Bridge: edge whose removal disconnects graph
  - Articulation point: vertex whose removal disconnects graph
  - Both use tin/lowlink technique

Shortest Path Variants:
  - Dijkstra: O((V+E) log V) for non-negative weights
  - 0-1 BFS: deque instead of priority queue O(V+E)
  - Dial's algorithm: when weights are small integers
  - Bellman-Ford: O(VE), detects negative cycles
  - Floyd-Warshall: O(V³), all-pairs shortest

Minimum Spanning Tree:
  - Kruskal: sort edges, DSU union, O(E log E)
  - Prim: similar to Dijkstra, O(E log V)
  - Boruvka: for dense graphs, parallelizable

Maximum Flow:
  - Ford-Fulkerson: O(E * maxflow)
  - Edmonds-Karp: BFS instead of DFS, O(VE²)
  - Dinic: O(V²E) in general, O(E√V) for unit capacities
  - Push-Relabel: O(V³), fast in practice

  Applications:
    - Bipartite matching (connect source → left → right → sink)
    - Minimum cut (max flow min cut theorem)
    - Vertex/edge-disjoint paths
    - Project selection (profit maximization)
    - Image segmentation
```

### 5.5 String Algorithms

```
Z-Algorithm:
  - Z-array: Z[i] = longest common prefix of s[0:] and s[i:]
  - Used for: pattern matching, longest palindrome prefix

  vector<int> z_function(string s) {
      int n = s.size();
      vector<int> z(n);
      for (int i = 1, l = 0, r = 0; i < n; i++) {
          if (i <= r) z[i] = min(r - i + 1, z[i - l]);
          while (i + z[i] < n && s[z[i]] == s[i + z[i]]) z[i]++;
          if (i + z[i] - 1 > r) l = i, r = i + z[i] - 1;
      }
      return z;
  }

KMP (Knuth-Morris-Pratt):
  - LPS array: longest proper prefix which is also suffix
  - O(n+m) pattern matching

  vector<int> prefix_function(string s) {
      int n = s.size();
      vector<int> pi(n);
      for (int i = 1; i < n; i++) {
          int j = pi[i-1];
          while (j > 0 && s[i] != s[j]) j = pi[j-1];
          if (s[i] == s[j]) j++;
```

```
            pi[i] = j;
        }
        return pi;
    }

Manacher's Algorithm:
  - Find all palindromic substrings in O(n)
  - d1[i] = odd-length palindrome radius at i
  - d2[i] = even-length palindrome radius at i

Rabin-Karp Rolling Hash:
  - Hash = Σ s[i] * p^i (mod M)
  - Update: hash = (hash - s[l] * p^l) * p_inv ... careful with positioning
  - Single hash may collide; use double hash for safety

Suffix Array & LCP:
  - Suffix array: sorted suffixes of string
  - LCP: longest common prefix between consecutive suffixes
  - O(n log n) construction (prefix doubling or SA-IS)
  - Used for: longest repeated substring, longest common substring, substring count
```

---

## 6. PHASE 4: ADVANCED TOPICS

### Duration: 6-8 weeks

### 6.1 Heavy-Light Decomposition

```
Purpose: Perform path queries on trees (sum, min, max, update)
  - Decompose tree into heavy paths
  - At most log n paths to query
  - Use segment tree over flattened node order

  Steps:
    1. DFS to compute subtree sizes and heavy child
    2. Assign head of each path
    3. Map nodes to segment tree positions
    4. Query path: climb from u to v, updating/querying segments

  int head[N], pos[N], parent[N], depth[N], heavy[N], sz[N];

  void decompose(int v, int h) {
      head[v] = h; pos[v] = curPos++;
      if (heavy[v] != -1) decompose(heavy[v], h);
      for (int u : adj[v])
          if (u != parent[v] && u != heavy[v])
              decompose(u, u);
  }

  ll query_path(int u, int v) {
      ll res = 0;
      while (head[u] != head[v]) {
          if (depth[head[u]] < depth[head[v]]) swap(u, v);
          res += seg.query(pos[head[u]], pos[u]); // sum over path segment
          u = parent[head[u]];
      }
      if (depth[u] > depth[v]) swap(u, v);
      res += seg.query(pos[u], pos[v]); // includes LCA
      return res;
  }
```

### 6.2 Centroid Decomposition

```
Purpose: Solve path-counting problems on trees (especially distance-based)
  - Recursively find centroid, process paths through it
  - O(n log n) total complexity

  Steps per recursive call:
    1. Find centroid (node where max subtree size ≤ total/2)
    2. Process all paths that pass through centroid
    3. Remove centroid, recurse on each connected component

  int find_centroid(int v, int p, int total) {
      int max_sub = 0;
      for (int u : adj[v]) {
          if (u != p && !removed[u]) {
              int sz_u = subtree_size(u, v);
              max_sub = max(max_sub, sz_u);
              if (sz_u > total/2) return find_centroid(u, v, total);
          }
      }
      return v;
  }
```

### 6.3 Link-Cut Tree

```
Purpose: Dynamic tree operations (link, cut, path queries)
  - Represents forest of rooted trees
  - Operations: makeRoot, findRoot, link, cut, path query
  - Uses splay trees for preferred paths
  - All operations O(log n) amortized

  Useful for problems:
    - Dynamic connectivity in forests
    - Path queries with edge updates
    - Minimum spanning tree with edge insertions
    - Dynamic subtree queries (with subtree queries extension)
```

### 6.4 Flow Algorithms Advanced

```
Dinic's Algorithm:
  struct Dinic {
      struct Edge { int v, rev; ll cap; };
      vector<vector<Edge>> graph;
      vector<int> level, ptr;

      Dinic(int n) : graph(n), level(n), ptr(n) {}

      void add_edge(int u, int v, ll cap) {
          graph[u].push_back({v, (int)graph[v].size(), cap});
          graph[v].push_back({u, (int)graph[u].size()-1, 0});
      }

      bool bfs(int s, int t) {
          fill(level.begin(), level.end(), -1);
          queue<int> q({s}); level[s] = 0;
          while (!q.empty()) {
              int u = q.front(); q.pop();
              for (auto& e : graph[u])
                  if (e.cap > 0 && level[e.v] == -1)
                      level[e.v] = level[u] + 1, q.push(e.v);
          }
```

```
        return level[t] != -1;
    }

    ll dfs(int u, int t, ll flow) {
        if (u == t) return flow;
        for (int& i = ptr[u]; i < (int)graph[u].size(); i++) {
            auto& e = graph[u][i];
            if (e.cap > 0 && level[e.v] == level[u] + 1) {
                ll pushed = dfs(e.v, t, min(flow, e.cap));
                if (pushed) {
                    e.cap -= pushed;
                    graph[e.v][e.rev].cap += pushed;
                    return pushed;
                }
            }
        }
        return 0;
    }

    ll max_flow(int s, int t) {
        ll flow = 0;
        while (bfs(s, t)) {
            fill(ptr.begin(), ptr.end(), 0);
            while (ll pushed = dfs(s, t, LLONG_MAX)) flow += pushed;
        }
        return flow;
    }
};

Min-Cost Max-Flow (Successive Shortest Path):
  - Use Bellman-Ford or potentials (Johnson's) for negative edges
  - Each augmenting path: shortest path in residual graph
  - Complexity: O(F * E log V) where F = flow

Hungarian Algorithm:
  - Assignment problem: minimum weight matching in bipartite graph
  - O(n³) implementation, n = size of matrix

int hungarian(vector<vector<int>>& a) {
    int n = a.size(), m = a[0].size();
    vector<int> u(n+1), v(m+1), p(m+1), way(m+1);
    for (int i = 1; i <= n; i++) {
        p[0] = i; int j0 = 0;
        vector<int> minv(m+1, INF);
        vector<bool> used(m+1, false);
        do {
            used[j0] = true;
            int i0 = p[j0], delta = INF, j1;
            for (int j = 1; j <= m; j++)
                if (!used[j]) {
                    int cur = a[i0-1][j-1] - u[i0] - v[j];
                    if (cur < minv[j]) minv[j] = cur, way[j] = j0;
                    if (minv[j] < delta) delta = minv[j], j1 = j;
                }
            for (int j = 0; j <= m; j++)
                if (used[j]) u[p[j]] += delta, v[j] -= delta;
                else minv[j] -= delta;
            j0 = j1;
        } while (p[j0] != 0);
        do {
            int j1 = way[j0];
```

```
            p[j0] = p[j1], j0 = j1;
        } while (j0);
    }
    return -v[0];
}
```

### 6.5 Convex Hull Trick & Li Chao Tree

```
Convex Hull Trick (Lines):
  - Maintain set of lines: y = m*x + b
  - Query max/min y for given x
  - Lines sorted by slope (monotonic) → deque, O(1) amortized
  - Arbitrary insertions/query → Li Chao tree

  struct Line {
      ll m, b; // y = m*x + b
      ll eval(ll x) { return m * x + b; }
  };

  // Monotonic slopes + monotonic queries (deque)
  struct CHT {
      deque<Line> dq;
      bool bad(Line l1, Line l2, Line l3) {
          return (l3.b - l1.b) * (l1.m - l2.m) <=
                 (l2.b - l1.b) * (l1.m - l3.m);
      }
      void add(Line l) {
          while (dq.size() >= 2 && bad(dq[dq.size()-2], dq.back(), l))
              dq.pop_back();
          dq.push_back(l);
      }
      ll query(ll x) {
          while (dq.size() >= 2 && dq[0].eval(x) >= dq[1].eval(x))
              dq.pop_front();
          return dq[0].eval(x);
      }
  };

Li Chao Tree (arbitrary lines, no constraints):
  struct LiChao {
      struct Line { ll m, b; ll eval(ll x) { return m*x + b; } };
      vector<Line> tree;
      int n;
      LiChao(int sz) {
          n = 1; while (n < sz) n <<= 1;
          tree.assign(2*n, {0, LLONG_MIN});
      }
      void add_line(Line line, int node, int l, int r) {
          int mid = (l + r) / 2;
          bool left = line.eval(l) > tree[node].eval(l);
          bool mid_left = line.eval(mid) > tree[node].eval(mid);
          if (mid_left) swap(tree[node], line);
          if (l == r) return;
          if (left != mid_left) add_line(line, 2*node, l, mid);
          else add_line(line, 2*node+1, mid+1, r);
      }
      ll query(int x, int node, int l, int r) {
          ll res = tree[node].eval(x);
          if (l == r) return res;
          int mid = (l + r) / 2;
          if (x <= mid) return max(res, query(x, 2*node, l, mid));
```

```
            else return max(res, query(x, 2*node+1, mid+1, r));
        }
    };
```

### 6.6 FFT & NTT

```
Fast Fourier Transform (FFT):
  - Multiply two polynomials in O(n log n)
  - Complex roots of unity
  - Cooley-Tukey algorithm (divide and conquer)

  using cd = complex<double>;
  const double PI = acos(-1);

  void fft(vector<cd>& a, bool invert) {
      int n = a.size();
      for (int i = 1, j = 0; i < n; i++) {
          int bit = n >> 1;
          for (; j & bit; bit >>= 1) j ^= bit;
          j ^= bit;
          if (i < j) swap(a[i], a[j]);
      }
      for (int len = 2; len <= n; len <<= 1) {
          double ang = 2 * PI / len * (invert ? -1 : 1);
          cd wlen(cos(ang), sin(ang));
          for (int i = 0; i < n; i += len) {
              cd w(1);
              for (int j = 0; j < len/2; j++) {
                  cd u = a[i+j], v = a[i+j+len/2] * w;
                  a[i+j] = u + v;
                  a[i+j+len/2] = u - v;
                  w *= wlen;
              }
          }
      }
      if (invert) for (cd& x : a) x /= n;
  }

  vector<int> multiply(vector<int>& a, vector<int>& b) {
      vector<cd> fa(a.begin(), a.end()), fb(b.begin(), b.end());
      int n = 1;
      while (n < a.size() + b.size()) n <<= 1;
      fa.resize(n); fb.resize(n);
      fft(fa, false); fft(fb, false);
      for (int i = 0; i < n; i++) fa[i] *= fb[i];
      fft(fa, true);
      vector<int> res(n);
      for (int i = 0; i < n; i++) res[i] = round(fa[i].real());
      return res;
  }

NTT (Number Theoretic Transform):
  - FFT over modular arithmetic (mod prime like 998244353 = 119*2^23 + 1)
  - Primitive root: 3 is a primitive root for 998244353
  - Avoid floating-point errors
```

### 6.7 Game Theory

```
Grundy Numbers (Nimbers):
  - mex (minimum excludant) of reachable states
  - Grundy(x) = mex{ Grundy(y) for all y reachable from x }
```

```
  - XOR of all component Grundy numbers determines winner
  - If XOR ≠ 0: first player wins

  Sprague-Grundy Theorem:
    - Any impartial combinatorial game = Nim heap
    - Grundy number = size of equivalent Nim heap
    - Winning position: XOR of Grundy numbers ≠ 0

Classic Games:
  - Nim: XOR of pile sizes ≠ 0 → first player wins
  - Nim variants: Moore's Nim, Wythoff's Game, Kayles
  - Subtraction Games: take 1..K from pile
  - Take-away games: Fibonacci Nim, Euclid's game

  Minimax Algorithm:
    - For two-player zero-sum games
    - Evaluate game tree: maximize for current player, minimize for opponent
    - Alpha-beta pruning: prune branches that cannot affect result
```

### 6.8 Polynomial & String Hashing

```
Double Hashing:
  - Two moduli to avoid collisions
  - Base between 131 and 91138233
  - Precompute powers for O(1) substring hash

  struct DoubleHash {
      vector<pair<ll,ll>> h, p;
      ll mod1 = 1e9+7, mod2 = 1e9+9, base = 91138233;

      DoubleHash(string s) {
          int n = s.size();
          h.resize(n+1); p.resize(n+1);
          p[0] = {1, 1};
          for (int i = 1; i <= n; i++) {
              p[i].first = p[i-1].first * base % mod1;
              p[i].second = p[i-1].second * base % mod2;
              h[i].first = (h[i-1].first * base + s[i-1]) % mod1;
              h[i].second = (h[i-1].second * base + s[i-1]) % mod2;
          }
      }

      pair<ll,ll> get(int l, int r) {
          ll h1 = (h[r].first - h[l].first * p[r-l].first % mod1 + mod1) % mod1;
          ll h2 = (h[r].second - h[l].second * p[r-l].second % mod2 + mod2) % mod2;
          return {h1, h2};
      }
  };
```

---

## 7. PHASE 5: EXPERT & MASTER LEVEL

### Duration: 8-12+ weeks

### 7.1 Advanced Data Structures

```
Treap (Cartesian Tree + Heap):
  - Random priority ensures balanced tree
  - Merge and Split operations
  - Can support: range queries, range updates, reverse, insert/delete
```

```
  struct Treap {
      int val, prio, sz;
      Treap *l, *r;
      Treap(int v) : val(v), prio(rand()), sz(1), l(NULL), r(NULL) {}
  };

  int size(Treap* t) { return t ? t->sz : 0; }
  void upd(Treap* t) { if (t) t->sz = 1 + size(t->l) + size(t->r); }

  void split(Treap* t, int key, Treap*& l, Treap*& r) {
      if (!t) { l = NULL; r = NULL; return; }
      if (size(t->l) >= key) {
          split(t->l, key, l, t->l);
          r = t;
      } else {
          split(t->r, key - size(t->l) - 1, t->r, r);
          l = t;
      }
      upd(t);
  }

  Treap* merge(Treap* l, Treap* r) {
      if (!l || !r) return l ? l : r;
      if (l->prio > r->prio) {
          l->r = merge(l->r, r);
          upd(l); return l;
      } else {
          r->l = merge(l, r->l);
          upd(r); return r;
      }
  }

Policy-Based Data Structures (GNU C++):
  #include <ext/pb_ds/assoc_container.hpp>
  #include <ext/pb_ds/tree_policy.hpp>
  using namespace __gnu_pbds;

  template<typename T>
  using ordered_set = tree<T, null_type, less<T>, rb_tree_tag,
                           tree_order_statistics_node_update>;

  // order_of_key(k): number of elements strictly less than k
  // find_by_order(k): k-th element (0-indexed)

  // For using as multiset:
  using ordered_multiset = tree<pair<int,int>, null_type, less<pair<int,int>>,
                                rb_tree_tag, tree_order_statistics_node_update>;

Dynamic Connectivity:
  - Offline: Divide and Conquer + DSU with rollback
  - Online: Euler Tour Tree (ETT) or Link-Cut Tree

Dynamic Segment Tree:
  - Nodes created on demand (for sparse arrays)
  - Point updates, range queries in O(log N) where N = coordinate range
  - Good when N is large (10^9) but updates are few

Persistent Segment Tree:
  - Version control for segment tree
  - Create new root for each version
  - O(log n) time and space per update
```

```
  - Used in: K-th number in range, static range queries with updates
```

### 7.2 Geometry & Computational Geometry

```
Primitives:
  struct Point { ll x, y; };
  ll cross(Point a, Point b) { return a.x*b.y - a.y*b.x; }
  ll dot(Point a, Point b) { return a.x*b.x + a.y*b.y; }
  ll dist2(Point a, Point b) {
      return (a.x-b.x)*(a.x-b.x) + (a.y-b.y)*(a.y-b.y);
  }

  // Orientation: 1 = ccw, -1 = cw, 0 = collinear
  int orient(Point a, Point b, Point c) {
      ll val = (b.y-a.y)*(c.x-b.x) - (b.x-a.x)*(c.y-b.y);
      if (val == 0) return 0;
      return (val > 0) ? 1 : -1;
  }

  // Point on segment?
  bool onSegment(Point p, Point a, Point b) {
      return orient(a, b, p) == 0 &&
             min(a.x,b.x) <= p.x && p.x <= max(a.x,b.x) &&
             min(a.y,b.y) <= p.y && p.y <= max(a.y,b.y);
  }

  // Line intersection
  Point lineIntersect(Point a, Point b, Point c, Point d) {
      ll a1 = b.y-a.y, b1 = a.x-b.x, c1 = a1*a.x + b1*a.y;
      ll a2 = d.y-c.y, b2 = c.x-d.x, c2 = a2*c.x + b2*c.y;
      ll det = a1*b2 - a2*b1;
      // parallel if det == 0
      return {(c1*b2 - c2*b1) / det, (a1*c2 - a2*c1) / det};
  }

Convex Hull:
  - Andrew's monotone chain: O(n log n)
  - Graham scan: O(n log n)

  vector<Point> convex_hull(vector<Point> pts) {
      sort(pts.begin(), pts.end(), [](Point a, Point b) {
          return a.x < b.x || (a.x == b.x && a.y < b.y);
      });
      vector<Point> hull;
      for (int phase = 0; phase < 2; phase++) {
          int start = hull.size();
          for (auto& p : pts) {
              while (hull.size() >= start + 2 &&
                     orient(hull[hull.size()-2], hull.back(), p) <= 0)
                  hull.pop_back();
              hull.push_back(p);
          }
          hull.pop_back();
          reverse(pts.begin(), pts.end());
      }
      return hull;
  }

  Closest Pair of Points:
    - Divide and conquer: O(n log n)
    - Sort by x, split, recurse, then check points within strip
```

```
  Rotating Calipers:
    - Find diameter of convex polygon
    - Minimum bounding rectangle
    - Distance between convex polygons
```

### 7.3 Complexity Theory

```
P vs NP:
  - P: problems solvable in polynomial time
  - NP: problems verifiable in polynomial time
  - NP-Complete: hardest problems in NP (SAT, 3-SAT, Vertex Cover, Hamiltonian Cycle)
  - NP-Hard: at least as hard as NP-Complete, not necessarily in NP
  - Whether P = NP is the biggest open problem in CS

Reductions:
  - Converting problem A to problem B
  - If A reduces to B and B is in P, then A is in P
  - If A reduces to B and A is NP-Complete, then B is NP-Hard

Approximation Algorithms:
  - Vertex Cover: 2-approximation (maximal matching)
  - Set Cover: (ln n)-approximation (greedy)
  - Traveling Salesman: 1.5-approximation (Christofides algorithm)
  - Knapsack: FPTAS (1+ε)-approximation

Heuristics for CP:
  - Genetic algorithms for some optimization problems
  - Simulated Annealing
  - Iterated Local Search
  - These rarely appear in standard CP contests (mostly in optimization contests)
```

---

## 8. PLATFORM STRATEGIES

### 8.1 Codeforces Strategy

```
Rating System:
  - Newbie: 0-1199
  - Pupil: 1200-1399
  - Specialist: 1400-1599
  - Expert: 1600-1899
  - Candidate Master: 1900-2099
  - Master: 2100-2299
  - International Master: 2300-2399
  - Grandmaster: 2400-2599
  - International Grandmaster: 2600-2999
  - Legendary Grandmaster: 3000+

Problem Rating Distribution:
  A: 800-1200 (easy, solve fast)
  B: 1100-1500 (implementation, some thinking)
  C: 1400-1800 (moderate difficulty)
  D: 1600-2100 (hard, often DP/graphs)
  E: 1900-2400 (advanced)
  F: 2100-2800 (very hard)

Contest Strategy:
  Phase 1 (0-30 min): Solve A and B as fast as possible
  Phase 2 (30-60 min): Work on C
```

```
  Phase 3 (60-120 min): C (if not solved) or D
  Phase 4 (120+ min): D/E (if time remains)

  Time allocation:
    - Div2 A: 3-5 min
    - Div2 B: 10-15 min
    - Div2 C: 20-40 min
    - Div2 D: 30-90 min

Upsolving:
  - After each contest, solve ALL problems you didn't solve
  - Read editorial, understand, implement from scratch
  - Wait 1 week then re-solve (spaced repetition)

Practice Plan:
  - Daily: 3 problems at current rating + 1 problem at +200 rating
  - Weekly: 2-3 virtual contests from 1-2 months ago
  - Weekly: Participate in live contest
  - Topic focus: 2-3 topics per week (e.g., DP + bit manipulation)
```

### 8.2 AtCoder Strategy

```
Rating System:
  - Gray (0-399): Beginner
  - Brown (400-799): Novice
  - Green (800-1199): Intermediate
  - Cyan (1200-1599): Advanced
  - Blue (1600-1999): Expert
  - Yellow (2000-2399): Master
  - Orange (2400-2799): Grandmaster
  - Red (2800+): Legend

Contest Types:
  - ABC (AtCoder Beginner Contest): Weekly, 7 problems (A-G)
    - A, B: very easy (like CF 800)
    - C, D, E: moderate (like CF 1200-1600)
    - F, G: hard (like CF 1800-2400)

  - ARC (AtCoder Regular Contest): Monthly, harder problems
  - AGC (AtCoder Grand Contest): Quarterly, very hard

AtCoder Specifics:
  - More math-heavy than CF
  - Many combinatorics, probability, number theory problems
  - Unique problems: constructives, interactive
  - Better quality problem statements
  - Slower rating inflation than CF

  Recommended for: improving math skills, constructive thinking
  ABC-C/D are excellent for Specialist-level growth
```

### 8.3 CodeChef Strategy

```
Rating System:
  - 1 Star: 0-1399
  - 2 Star: 1400-1599
  - 3 Star: 1600-1799
  - 4 Star: 1800-1999
  - 5 Star: 2000-2199
  - 6 Star: 2200-2499
  - 7 Star: 2500+
```

```
Contest Types:
  - Starters: Weekly, Wednesday
  - Cook-Off: Monthly, Sunday (now merged with Starters)
  - Lunchtime: Monthly, Saturday (now merged with Starters)

CodeChef Specifics:
  - More implementation-heavy than CF
  - Longer problem statements
  - More story-based problems
  - Good for: building stamina for long contests
  - Many problems are math-heavy (modular arithmetic, combinatorics)
```

### 8.4 LeetCode Strategy

```
Rating System:
  - 1400-1599: Novice
  - 1600-1799: Apprentice
  - 1800-1999: Specialist
  - 2000-2199: Expert
  - 2200-2399: Guardian
  - 2400+: Legend

Weekly Contest (every Saturday night):
  4 problems: Easy, Medium, Medium, Hard
  Time: 90 minutes

  Strategy:
    - Q1: 5-10 min (simple)
    - Q2: 15-20 min (moderate)
    - Q3: 20-40 min (medium-hard)
    - Q4: 30-60 min (hard)

Biweekly Contest (alternate Saturdays):
  Same format as Weekly

Use LC for:
  - Interview preparation (company-tagged problems)
  - Easy-to-moderate problem practice
  - Building speed on common patterns
  - Not: advanced CP techniques (FFT, flows, etc.)
```

---

## 9. CONTEST STRATEGY & TEMPLATES

### 9.1 Pre-Contest Preparation

```
Before Contest (30 minutes):
  1. Setup IDE: open template, configure input/output
  2. Clone template and rename with problem names
  3. Refresh algorithms on current weak topics
  4. Set up timer/stress test tool
  5. Ensure good internet connection
  6. Silence phone, eliminate distractions

Environment Checklist:
  [ ] Template file ready (fast I/O, debug macros)
  [ ] Local judge/test case runner configured
  [ ] Stress test script ready
  [ ] Browser tabs: platform, CP-Algorithms, personal notes
```

```
  [ ] Paper and pen for notes/diagrams
```

### 9.2 During Contest

```
Reading Problem Statements:
  1. Read statement completely (2-3 times)
  2. Identify input/output format
  3. Check constraints (what complexity is needed?)
  4. Look at sample test cases
  5. Identify edge cases
  6. Determine algorithm/approach before coding

Decision Flow:
  - Can I brute force? (check constraints)
  - Is there a greedy solution?
  - Does DP apply? (optimal substructure + overlapping subproblems)
  - Does a graph model work?
  - Is it a known problem pattern?

  15-minute rule: If no progress in 15 min, switch problems or read editorial after
   ↪  contest

Debugging During Contest:
  - Print intermediate values
  - Check off-by-one errors
  - Verify integer overflow (use long long)
  - Test edge cases: min/max values, empty input
  - Corner cases: n=1, n=2, all same, already sorted
  - Compare with brute force for small n

Time Management:
  - Don't spend more than 30 min on a problem you're stuck on
  - If stuck: move to another problem, come back later
  - The mind works subconsciously; a break might help
  - Read all problems first before diving deep
  - Sometimes an easier problem (C) might be after a hard one (B)
```

### 9.3 Post-Contest

```
Immediately After:
  1. Write down what went well and what went wrong
  2. Identify which problems I should have solved
  3. Start upsolving (don't wait!)

Upsolving Steps:
  1. Try to solve without editorial (15-30 min)
  2. Read editorial
  3. Understand the logic (visualize, draw)
  4. Implement from scratch (no peeking)
  5. Compare with official implementation
  6. Add to CP notes / journal

Spaced Repetition:
  - Re-solve after 1 day
  - Re-solve after 1 week
  - Re-solve after 1 month
  - Use Anki for algorithm flash cards
```

### 9.4 Template Code

```
#include <bits/stdc++.h>
```

```
using namespace std;

typedef long long ll;
typedef vector<int> vi;
typedef vector<ll> vll;
typedef vector<vi> vvi;
typedef pair<int, int> pii;
typedef vector<pii> vpii;

#define all(x) (x).begin(), (x).end()
#define sz(x) (int)(x).size()
#define pb push_back
#define mp make_pair
#define fi first
#define se second

const int MOD = 1e9 + 7;
const int INF = 1e9;
const ll LINF = 1e18;
const double EPS = 1e-9;

// Fast I/O
void fastIO() {
    ios_base::sync_with_stdio(false);
    cin.tie(NULL);
    cout.tie(NULL);
}

// Debug
template <typename T>
void print(T t) { cerr << t; }
template <typename T, typename... Args>
void print(T t, Args... args) { cerr << t << " "; print(args...); }
#ifdef LOCAL
    #define debug(...) cerr << "[" << #__VA_ARGS__ << "] = ", print(__VA_ARGS__), cerr <<
     ↪  '\n'
    #define debuga(arr, n) for(int i=0;i<n;i++) cerr<<arr[i]<<" \n"[i==n-1]
#else
    #define debug(...) 42
    #define debuga(...) 42
#endif

// Math
ll binpow(ll a, ll b, ll mod) {
    ll res = 1;
    while (b) {
        if (b & 1) res = res * a % mod;
        a = a * a % mod;
        b >>= 1;
    }
    return res;
}

ll gcd(ll a, ll b) { return b ? gcd(b, a % b) : a; }
ll lcm(ll a, ll b) { return a / gcd(a, b) * b; }

// For debugging small arrays
string to_string(vi v) {
    string s = "[";
    for (int i = 0; i < sz(v); i++) {
        s += to_string(v[i]);
```

```
        if (i < sz(v)-1) s += ", ";
    }
    return s + "]";
}

// Main solve function
void solve() {
    // solution here
}

int main() {
    fastIO();
    int t = 1;
    cin >> t;
    while (t--) {
        solve();
    }
    return 0;
}
```

### 9.5 Stress Testing

```
// stress_test.cpp
// Compare your solution with a brute force on random inputs

#include <bits/stdc++.h>
using namespace std;

// Your solution
int solve_fast(vector<int>& arr) {
    // your algorithm
}

// Brute force (guaranteed correct)
int solve_brute(vector<int>& arr) {
    // correct but slow algorithm
}

int main() {
    srand(time(0));
    for (int test = 1; test <= 10000; test++) {
        int n = rand() % 10 + 1;
        vector<int> arr(n);
        for (int i = 0; i < n; i++)
            arr[i] = rand() % 20;

        int ans1 = solve_fast(arr);
        int ans2 = solve_brute(arr);

        if (ans1 != ans2) {
            cout << "Test " << test << " failed!\n";
            cout << "n = " << n << "\narr = ";
            for (int x : arr) cout << x << " ";
            cout << "\nFast: " << ans1 << " Brute: " << ans2 << "\n";
            return 0;
        }
        if (test % 1000 == 0)
            cout << "Passed " << test << " tests\n";
    }
    cout << "All tests passed!\n";
    return 0;
```

```
}
```

---

## 10. 100+ MUST-SOLVE PROBLEMS

### 10.1 Arrays & Prefix Sums (10 problems)

| Problem | Platform | Rating | Technique |
|---|---|---|---|
| Maximum Subarray | LeetCode 53 | Easy | Kadane's algorithm |
| Subarray Sum Equals K | LeetCode 560 | Medium | Prefix sum + hashmap |
| Range Sum Query 2D | LeetCode 304 | Medium | 2D prefix sum |
| Maximum Product Subarray | LeetCode 152 | Medium | Kadane's variant |
| Product of Array Except Self | LeetCode 238 | Medium | Prefix/suffix products |
| Subarray with GCD K | CF 1765N | 1600 | GCD property |
| Count Subarrays with Fixed Bounds | LC 2444 | Hard | Sliding window |
| Maximum Points You Can Obtain | LC 1423 | Medium | Prefix + sliding |
| Continuous Subarray Sum | LC 523 | Medium | Prefix sum + modulo |
| Minimum Size Subarray Sum | LC 209 | Medium | Two pointer |

### 10.2 Binary Search (10 problems)

| Problem | Platform | Rating | Technique |
|---|---|---|---|
| Search in Rotated Sorted Array | LC 33 | Medium | Binary search with rotation |
| Find Min in Rotated Sorted | LC 153 | Medium | Binary search |
| Aggressive Cows | SPOJ | Medium | BS on answer |
| Book Allocation | GFG | Hard | BS on answer |
| Koko Eating Bananas | LC 875 | Medium | BS on answer |
| Capacity to Ship Packages | LC 1011 | Medium | BS on answer |
| Median of Two Sorted Arrays | LC 4 | Hard | BS on smaller array |
| Kth Element of Two Sorted | GFG | Hard | BS on smaller |
| Split Array Largest Sum | LC 410 | Hard | BS on answer + greedy |
| Find Peak Element | LC 162 | Medium | BS on non-sorted |

### 10.3 Stacks & Queues (10 problems)

| Problem | Platform | Rating | Technique |
|---|---|---|---|
| Next Greater Element I | LC 496 | Easy | Monotonic stack |

| Problem | Platform | Rating | Technique |
|---|---|---|---|
| Largest Rectangle in Histogram | LC 84 | Hard | Monotonic stack |
| Maximal Rectangle | LC 85 | Hard | Histogram extension |
| Trapping Rain Water | LC 42 | Hard | Stack / two pointer |
| Sliding Window Maximum | LC 239 | Hard | Deque |
| Remove K Digits | LC 402 | Medium | Monotonic stack |
| Sum of Subarray Minimums | LC 907 | Medium | Monotonic stack |
| Minimum Remove Parentheses | LC 1249 | Medium | Stack |
| Asteroid Collision | LC 735 | Medium | Stack simulation |
| Online Stock Span | LC 901 | Medium | Monotonic stack |

### 10.4 Linked Lists (8 problems)

| Problem | Platform | Rating | Technique |
|---|---|---|---|
| Reverse Linked List | LC 206 | Easy | Iterative + recursive |
| Merge Two Sorted Lists | LC 21 | Easy | Two pointers |
| LRU Cache | LC 146 | Medium | DLL + hashmap |
| Reverse Nodes in K-Group | LC 25 | Hard | Reverse in groups |
| Copy with Random Pointer | LC 138 | Medium | Interleaving |
| Detect Cycle | LC 141 | Easy | Floyd's algorithm |
| Palindrome Linked List | LC 234 | Medium | Reverse half |
| Intersection of Two Lists | LC 160 | Easy | Two pointers |

### 10.5 Trees & BST (12 problems)

| Problem | Platform | Rating | Technique |
|---|---|---|---|
| Binary Tree Inorder Traversal | LC 94 | Easy | Morris traversal |
| Max Path Sum | LC 124 | Hard | DFS tree |
| LCA of Binary Tree | LC 236 | Medium | DFS |
| Serialize & Deserialize | LC 297 | Hard | BFS / DFS |
| Binary Tree from Inorder/Preorder | LC 105 | Medium | Recursion |
| Validate BST | LC 98 | Medium | Inorder + range |
| Kth Smallest in BST | LC 230 | Medium | Inorder iterative |
| Flatten BT to LinkedList | LC 114 | Medium | Moris-like |
| All Nodes Distance K | LC 863 | Medium | Graph BFS on tree |
| Diameter of BT | LC 543 | Easy | DFS height |
| Balanced BT | LC 110 | Easy | DFS height check |
| Maximum Width of BT | LC 662 | Medium | Level order |

### 10.6 Graphs (15 problems)

| Problem | Platform | Rating | Technique |
|---|---|---|---|
| Number of Islands | LC 200 | Medium | DFS/BFS |
| Course Schedule | LC 207 | Medium | Topological sort |
| Dijkstra's Algorithm | GFG | Medium | Shortest path |

| Problem | Platform | Rating | Technique |
|---|---|---|---|
| Bellman-Ford Algorithm | GFG | Medium | Negative edges |
| Flood Fill | LC 733 | Easy | DFS |
| Pacific Atlantic Water Flow | LC 417 | Medium | DFS from edges |
| Word Ladder | LC 127 | Hard | BFS shortest path |
| Clone Graph | LC 133 | Medium | DFS + hashmap |
| Alien Dictionary | LC 269 | Hard | Topological sort |
| Cheapest Flights Within K Stops | LC 787 | Medium | Bellman-Ford |
| Network Delay Time | LC 743 | Medium | Dijkstra |
| Min Cost to Connect Points | LC 1584 | Medium | Prim's MST |
| Redundant Connection | LC 684 | Medium | DSU |
| Accounts Merge | LC 721 | Medium | DSU |
| Evaluate Division | LC 399 | Medium | Graph DFS |

### 10.7 Dynamic Programming (20 problems)

| Problem | Platform | Rating | Technique |
|---|---|---|---|
| Climbing Stairs | LC 70 | Easy | 1D DP |
| Coin Change | LC 322 | Medium | Unbounded knapsack |
| Longest Increasing Subsequence | LC 300 | Medium | O(n log n) patience |
| Longest Common Subsequence | LC 1143 | Medium | 2D DP |
| Edit Distance | LC 72 | Medium | String DP |
| 0/1 Knapsack | GFG | Medium | Classic DP |
| Rod Cutting | GFG | Medium | Unbounded knapsack |
| Burst Balloons | LC 312 | Hard | Interval DP |
| Palindrome Partitioning II | LC 132 | Hard | DP + palindrome check |
| Word Break | LC 139 | Medium | DP on string |
| House Robber | LC 198 | Medium | 1D DP |
| Maximum Profit Job Scheduling | LC 1235 | Hard | DP + binary search |
| Stone Game | LC 877 | Medium | Interval DP |
| Partition Equal Subset Sum | LC 416 | Medium | Subset sum |
| Target Sum | LC 494 | Medium | Subset sum variant |
| Cherry Pickup | LC 741 | Hard | 3D DP |
| Minimum Falling Path Sum | LC 931 | Medium | Grid DP |
| Count Square Submatrices | LC 1277 | Medium | Square DP |
| Unique Paths | LC 62 | Medium | Grid DP |
| Best Time to Buy/Sell Stock IV | LC 188 | Hard | DP with states |

### 10.8 Bit Manipulation (8 problems)

| Problem | Platform | Rating | Technique |
|---|---|---|---|
| Single Number | LC 136 | Easy | XOR |
| Single Number II | LC 137 | Medium | Bit counting |
| Single Number III | LC 260 | Medium | XOR + bit isolation |
| Power of Two | LC 231 | Easy | x & (x-1) |
| Count Bits | LC 338 | Easy | DP + LSB |
| Maximum XOR of Two Numbers | LC 421 | Medium | Trie |
| Subsets | LC 78 | Medium | Bitmask |
| Find Duplicate Number | LC 287 | Medium | Floyd + bit |

### 10.9 Number Theory & Combinatorics (10 problems)

| Problem | Platform | Rating | Technique |
|---|---|---|---|
| Count Primes | LC 204 | Medium | Sieve |
| nCr mod p | SPOJ/CF | Medium | Precompute factorial |
| Modular Exponentiation | GFG | Easy | binpow |
| Euler Totient | SPOJ ETF | Medium | Totient |
| Linear Sieve | CF 230B | 1300 | SPF |
| Divisor Count | CF 1332B | 1400 | Divisor function |
| Chinese Remainder | SPOJ CRT | Hard | CRT |
| Lucas Theorem | CF 1617C | 1700 | Lucas |
| Fibonacci with Matrix | CF 222E | 1800 | Matrix exponentiation |
| Mobius Function | CF 1559E | 1900 | Linear sieve |

### 10.10 Advanced (15+ problems)

| Problem | Platform | Rating | Technique |
|---|---|---|---|
| Range Sum Query | CF 1304C | 1600 | Segment tree |
| Inversion Count | SPOJ INVCNT | Medium | BIT |
| K-th Number in Range | SPOJ KQUERY | Hard | Persistent segtree |
| Dynamic Inversions | CF 1093E | 2100 | BIT + Treap |
| LCA in Tree | CF 208E | 1900 | Binary lifting |
| Bridges in Graph | CF 118E | 1800 | Tarjan |
| Strongly Connected Components | CF 427C | 1600 | Kosaraju |
| Minimum Cut | SPOJ MINCUT | Hard | Dinic |
| Max Flow | SPOJ FASTFLOW | Hard | Dinic |
| Convex Hull | CF 191C | 1900 | CHT |
| FFT Multiplication | SPOJ POLYMUL | Hard | FFT |
| Edit Distance with D&C DP | CF 321C | 2200 | D&C DP |
| Heavy-Light Decomposition | CF 342E | 2100 | HLD |
| Centroid Decomposition | CF 321C | 2200 | Centroid |
| Suffix Array | CF 123D | 2100 | Suffix array + LCP |

## 11. RATING TIERS & SKILLS TABLE

| Rating | Title | What You Should Know | Contest Performance |
|---|---|---|---|
| 0-800 | Newbie | Basic syntax, arrays, loops | Solve Div2A |
| 800-1000 | Newbie+ | STL containers, simple greedy, brute force | Solve A fast, sometimes B |
| 1000-1200 | Pupil | Prefix sums, two pointers, binary search, simple DP | Solve AB, think on C |
| 1200-1400 | Pupil+ | DFS/BFS, basic DP (knapsack, LIS), number theory basics, combinatorics | Solve ABC sometimes, B fast |
| 1400-1600 | Specialist | Advanced DP, segment tree, BIT, Dijkstra, DSU, modular arithmetic | Solve ABC consistently |
| 1600-1800 | Expert | Lazy propagation, binary lifting, flows (Dinic), MST variants, game theory | Solve ABCD, sometimes E |
| 1800-2000 | Candidate Master | HLD, centroid, treap, advanced flows, FFT, suffix structures | Solve ABCDE in Div2 |
| 2000-2200 | Master | LCT, polynomial algorithms, NTT, advanced geometry | Consistent Top 100 Div2 |
| 2200-2400 | International Master | Research-level algorithms, deep intuition | Div1 competitor |
| 2400+ | Grandmaster | Can solve anything in contest time | Div1 regular, World-class |

### Equivalent Ratings Across Platforms

| Codeforces | AtCoder | CodeChef | LeetCode |
|---|---|---|---|
| 1200 (Pupil) | 800 (Brown) | 2 Star | 1800 |
| 1400 (Specialist) | 1000 (Green) | 3 Star | 2000 |
| 1600 (Expert) | 1200 (Cyan) | 4 Star | 2200 |
| 1900 (CM) | 1600 (Blue) | 5 Star | 2400 |
| 2100 (Master) | 2000 (Yellow) | 6 Star | 2500 |
| 2400 (GM) | 2400 (Orange) | 7 Star | 2700 |

---

## 12. RESOURCES & TOOLS

### Must-Visit Websites

```
Core References:
  - CP-Algorithms.com ★★ THE reference for algorithms
  - USACO Guide (usaco.guide) ★ Structured DSA progression
  - AtCoder Library (ACL) ★ Production-quality implementations
  - C++ Reference (cppreference.com) STL documentation
```

```
Problem Sets & Lists:
  - CSES Problem Set: 300 problems, excellent for practice
  - A2OJ Ladders: Rating-based problem lists (outdated but still good)
  - Codeforces Problemset: Filter by rating + tags
  - AtCoder Problems: Problem recommendation engine
  - CF Tracker: Track your progress, find problems at right level

Visualization:
  - Visualgo.net ⋆ Algorithm visualizations
  - Algorithm Visualizer: Interactive algorithms
  - USFCA Visualization: Sorting, BST, etc.

Forums & Discussion:
  - Codeforces Blogs ⋆ Best CP community content
  - Codeforces EDU section: Structured courses
  - Math StackExchange: For math heavy problems
```

### Must-Follow People

```
Codeforces Legends:
  - tourist (Gennady Korotkevich): The GOAT, watch screencasts
  - Errichto (Kamil Debowski): Streams, problem solving
  - SecondThread (David Harmeyer): CP tutorials
  - Colin Galen (galencolin): Intuition-based CP content
  - Benq (Benjamin Qi): Best C++ templates
  - tmwilliamlin: Rating improvement guides
  - aryanc403: Blog tutorials
  - ecnerwala: Top-level solver, streams
  - neal_wu: Clear explanations

AtCoder Stars:
  - chokudai: AtCoder admin
  - tourist: Also dominates AtCoder
  - maspy: Many tutorial blog posts

YouTube:
  - Errichto: Algorithms, CP tips
  - Colin Galen: CP, DSA intuition
  - William Lin: CF contest reviews
  - Love Babbar: DSA for placements (Hindi)
  - Priyansh Agarwal: CP guide, tips
  - SecondThread: Advanced CP algorithms
```

### Essential Books

```
- "Competitive Programming" by Halim & Halim ⋆⋆
  The CP bible. Covers both fundamentals and advanced topics.
  4th edition contains extensive content.

- "Competitive Programmer's Handbook" by Antti Laaksonen ⋆⋆
  Free PDF, excellent for beginners to intermediate.
  Covers essential topics clearly.

- "Guide to Competitive Programming" by Laaksonen
  Same as CP Handbook, published by Springer.

- "CP Algorithms" (cp-algorithms.com)
  Online, free, community-maintained encyclopedia.

- "Introduction to Algorithms" (CLRS)
```

```
  Reference only. Too theoretical for CP but good for deep understanding.

- "Cracking the Coding Interview"
  For interview-focused DSA practice (not CP-focused).
```

### Tools & Extensions

```
Browser Extensions:
  - Competitive Companion: Auto-fetch problem test cases
  - CF-Predictor: Estimate rating change during contest
  - Carrot: In-contest performance tracker
  - Codeforces Enhancer: Theme, features, statistics
  - LeetCode CN Extension: Company tags, difficulty rating
  - LeetCode Rating Predictor: Estimate contest rating

Code Editors:
  - VS Code with CPH extension ⭐
  - CP Editor (standalone, lightweight)
  - Sublime Text (fast, minimal)
  - Vim (for power users)

Libraries & Tools:
  - cptemplate: Pre-written algorithm template
  - stress-test: Compare with brute force
  - CP Booster: Generate problem sets
  - CF Tool: Command-line CF interface
  - AtCoder CLI: Submit from terminal

Productivity:
  - Anki: Spaced repetition for algorithms
  - Obsidian / Notion: CP notes, journal
  - Time tracking: RescueTime / Toggl
```

### ICPC Preparation

```
Team Formation:
  - 3 members with complementary strengths
  - Ideally: Implementation specialist + Algorithm specialist + Debugger
  - Practice coding on one computer (ICPC format)

ICPC Topics (by frequency):
  High: Data structures, DP, graphs, math, strings
  Medium: Geometry, combinatorics, number theory
  Low: Game theory, flows, FFT

ICPC Resources:
  - ICPC World Finals problems (past years)
  - ICPC Archive (Kattis)
  - Codeforces Gym (past regionals)
  - ICPC Notebook: 25-page reference sheet

Practice Routine:
  - 3-hour team practice weekly (simulate contest)
  - Individual: 5+ hours/week on weak topics
  - Review past regionals: solve A-F
```

---

## FINAL WORDS

**CP is a marathon, not a sprint.** Progress is measured in months and years, not days. The rating graph of every grandmaster shows plateaus and breakthroughs.

**Consistency beats intensity.** 3 hours every day beats 10 hours on weekends. Build the habit of solving at least 1 problem daily, no exceptions.

**Editorials are not cheating.** Understanding someone else's solution is how you learn. The skill is in comprehension, memorization, and reapplication.

**You will feel stuck.** Months-long plateaus are normal. Break through by: switching platforms, studying editorial collections, revisiting fundamentals, or taking a 2-day break.

**The community is your greatest resource.** Read others' solutions, write blog posts, participate in discussions, teach juniors. The more you engage, the faster you grow.

**Your goal is not rating — it's learning.** The rating follows the skill. Focus on understanding deeply, and the numbers will take care of themselves.

**The only bad practice is no practice.** Short on time? Solve 1 problem in 30 min. No time today? Read 1 editorial. No zero days.